

Hoofdstuk 1 Binair Rekenen voor Informatici

1.1

1. 11
2. 45
3. 103
4. 150
5. 121
6. 266

1.2

1. 11001
2. 111111
3. 1100011
4. 11001000
5. 101000111

1.3

1. $101 + 110 = 1011$
2. $11011 + 1010 = 100101$
3. $1111 + 10101 = 100100$
4. $111111 + 101010 = 1101001$
5. $1010101 + 1100110 = 10111011$

$11011 + 11011 = 110110$ $101010 + 101010 = 1010100$

Uitdaging #1: $1\ 0000\ 0100$ Uitdaging #2: $26 * 3 = 110100$

1.4

1. $2^8 = 256$, $2^{16} = 65.536$, $2^{32} = 4.294.967.296$
2. 1000 1101
3. 1100 1011
4. 0111 1000
5. 0101 0110
6. $26 - 15 = 0001\ 1010 - 0000\ 1111 = 0001\ 1010 + 1111\ 0001 = 0000\ 1011 = 11_d$
7. $-31 - 6 = 1110\ 0001 - 0000\ 0110 = 1110\ 0001 + 1111\ 1010 = 1101\ 1011 = -37$
8. foute opgave... past niet in 8 bits 2complement >128

1.5

1. 0x73
2. 0x35
3. 0x88

- 4. 0xAA
- 5. 0b11111010
- 6. 0b11001010
- 7. 0b11011110
- 8. 0b01011010

Hoofdstuk 2

2.1

a. - b. bijvoorbeeld: geen accenten, niet echt internationaal, geen emojis c. UTF d. uit 1 tot 4 bytes (in het geval van UTF 8)

2.2

•

2.3 - 2.6

Zie uitwerkingen van leerlingen

2.7

4 NAND

S1	S2	R
0	0	1
0	1	1
1	0	1
1	1	0

5 NOR

S1	S2	R
0	0	1
0	1	0
1	0	0
1	1	0

6 XNOR

S1	S2	R
0	0	1

S1	S2	R
0	1	0
1	0	0
1	1	1

7 Schakeling

$R = (S1 \ \&\& \ S2) \ \text{XOR} \ S2$

S1	S2	S1 && S2	R
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

2.8

Zie vorige tabel 7 Schakeling

2.9

Een uitgang voor het resultaat, een uitgang voor de carry.

2.10

1 half adder tabel

S1	S2	R1	R2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$R1 = S1 \ \text{XOR} \ S2$ en $R2 = S1 \ \text{AND} \ S2$

- 2. Er wordt geen carry van de vorige optelling meegenomen/komt niet in de waarheidstabel voor.
- 3. Carry van de vorige optelling meenemen

2.11

•

2.12

Waarheidstabel met 16 entries voor 4 variables. Geen resultaat kolom.

Verdiepende opdrachten

1. NAND game
2. S1 OR S2 (De Morgan wet)
3. $\text{NOT}((S1 \text{ XNOR } S2) \text{ AND } (S1 \text{ OR } S2)) == S1 \text{ NAND } S2$
4.
 - o

Hoofdstuk 3 (2p)

3.1

1. Kast
2. DVD drive
3. Harde schijf
4. Geheugen
5. Moederbord
6. Geluidskaart
7. Grafische kaart
8. CPU/processor/koeling
9. PSU

3.2 (4p)

1. Processor opzoeken
2. Redenering over kracht processor: leeftijd, aantal cores, klok frequentie
3. Voordeel: snellere berekeningen, meer taken tegelijk. Nadeel: prijs, energieverbruik, warmte
4. Overklokken: sneller, warmer
5. Meerdere programma's tegelijk: drukker processor, dus warmer
6. Koeltechniek: passieve koeling. Geen geluid, minder effectief. Actieve koeling met fan: geluid, effectiever. Waterkoeling: erg effectief, water in je computer
7. De wet van Moore zal niet voor altijd gelden: quantum tunnel effect.
8. CIR, MAR, MDR: Current instruction register, Memory Address Register en Memory Data Register. CIR op de CPU: dan kan de instructie sneller gedecodeerd worden. Geen verkeer op databus. MAR: address voor de databus, MDR: de inhoud van of naar het address in het MAR

3.3 (3p)

1. SSD valt in de Harddisk plek. Na de RAM, voor de offline storage (tapes etc)
2. Werkgeheugen is erg kostbaar om een vergelijkbare hoeveelheid opslag te hebben. Bovendien is het vluchtig geheugen
3.
 - o
4. 64 bits architectuur heeft een 64 bits databus, waardoor meer RAM geadresseerd kan worden.
5. Het werkgeheugen raakt vol

6. Er kunnen geen nieuwe programma's gerund worden of er wordt virtual memory ingezet.
7. SAM
8. Moeilijke vraag...
9.
 - o

Hoofdstuk 4

4.1.1

Berekening ontleden (zie oefening 1)

operator 1: *, operanden: 2 en 10, antwoord: 20

operator 2: +, operanden: 4 en 20, antwoord: 24

Registers kiezen

register instellen: R1 = 2

instructie: `MOV R1, #2`

Instructies opschrijven

instructie 1: `MUL R1, #10 // 2 * 10`

instructie 2: `ADD R1, #4 // 20 + 4`

4.1.2

operator 1: *, operanden: 12 en 2, antwoord: 24

operator 2: +, operanden: 16 en 24, antwoord: 40

Registers kiezen

register instellen: R1 = 12

instructie: `MOV R1, #12`

Instructies opschrijven

instructie 1: `MUL R1, #2 // 12 * 2`

instructie 2: `ADD R1, #16 // 24 + 16`

4.1.3

operator 1: +, operanden: 16 en 12, antwoord: 28

operator 2: %, operanden: 28 en 3, antwoord: 1

Registers kiezen

register instellen: $R1 = 16$

instructie: `MOV R1, #16`

Instructies opschrijven

instructie 1: `ADD R1, #12 // 16 + 12`

instructie 2: `MOD R1, #3 // 28 % 3`

4.1.4

operator 1: *, operanden: 2 en 3, antwoord: 6

operator 2: *, operanden: 6 en 5, antwoord: 30

Registers kiezen

register instellen: $R1 = 2$

instructie: `MOV R1, #2`

Instructies opschrijven

instructie 1: `MUL R1, #3 // 2 * 3`

instructie 2: `MUL R1, #5 // 6 * 5`

4.1.5

operator 1: *, operanden: 14 en 2, antwoord: 28

operator 2: *, operanden: 28 en 3, antwoord: 84

operator 3: -, operanden: 84 en 15, antwoord: 69

Registers kiezen

register instellen: $R1 = 14$

instructie: `MOV R1, #14`

Instructies opschrijven

instructie 1: `MUL R1, #2 // 14 * 2`

instructie 2: `MUL R1, #3 // 28 * 3`

instructie 3: `SUB R1, #15 // 84 - 15`

4.1.6

operator 1: *, operanden: 4 en 4, antwoord: 16

operator 2: *, operanden: 16 en 6, antwoord: 96

operator 3: %, operanden: 96 en 2, antwoord: 0

Registers kiezen

register instellen: R1 = 4

instructie: **MOV R1, #4**

Instructie opschrijven

instructie 1: **MUL R1, #4 // 4 * 4**

instructie 2: **MUL R1, #6 // 16 * 6**

instructie 3: **DIV R1, #2 // 96 % 2**

4.2.a Machinetaal decoden

00110 101 00101010

instructiedeel		
opcode	00110	&
Rsd	101	5
imm8	00101010	170
betekenis	R5 = R5 & 170	
resultaat	\$10_d\$, want 0001 1011 & 1010 1010 == 0000 1010	

4.2.b Machinetaal decoden

00011 110 00101010

instructiedeel		
opcode	00011	-
Rsd	110	6
imm8	00101010	170
betekenis	R6 = R6 - 170	
resultaat	\$-121_d\$ want 49-170=-121	

4.2.c Machinetaal decoden

00001 010 00000100

instructiedeel		
opcode	00001	%

instructiedeel		
Rsd	010	2
imm8	00000100	4
betekenis		$R2 = R2 \% 4$
resultaat		\$3_d\$ want $11 \% 4 = 3$

4.2.d Machinetaal decoden

00010 100 00011000

instructiedeel		
opcode	00010	+
Rsd	100	4
imm8	00011000	24
betekenis		$R4 = R4 + 24$
resultaat		\$31_d\$ want $7 + 24 = 31$

4.2.e Machinetaal decoden

00111 011 00001110

instructiedeel		
opcode	00111	OR
Rsd	011	3
imm8	00001110	14
betekenis		$R3 = R3 \parallel 14$
resultaat		\$30_d\$ want $0001\ 0100 \ \backslash \mid \mid \ 0000\ 1110 = 0001\ 1110 = \$30_d\$$

4.2.f Machinetaal decoden

00001 001 00000101

instructiedeel		
opcode	00001	%
Rsd	001	1
imm8	0000 0101	5
betekenis		$R1 = R1 \% 5$

instructiedeel	
resultaat	1 want $16 \% 5 = 1$

4.3 Machinetaal schrijven

	opg	asm	Bin	Hex
a	7%3	MOV R1, #7	00101 001 0000 0111	0x2907
		MOD R1, #3	00001 001 0000 0011	0x0903
b	11+14	MOV R1, #11	00101 001 0000 1011	0x290B
		ADD R1, #14	00010 001 0000 1110	0x110E
c	49-25	MOV R1, 49	00101 001 0011 0001	0x2931
		SUB R1, #25	00011 001 0001 1001	0x1919
d	20%6	MOV R1, #20	00101 001 0001 0100	0x2914
		MOD R1, #6	00001 001 0000 0110	0x0906
e	16+16	MOV R1, #16	00101 001 0001 0000	0x2910
		ADD R1, #163	00010 001 0001 0000	0x1110
f	8&29	MOV R1, #8	00101 001 0000 1000	0x2908
		AND R1, #29	00110 001 0001 1101	0x311D

4.4 Machinetaal decoderen

instructie	opcode	rsd	imm8	asm
00101 011 00000111	00101	011	0000 0111	MOV R3, #7
00010 011 00001111	00010	011	0000 1111	ADD R3, #15
00001 011 00000100	00001	011	0000 0100	MOD R3, #4
00101 101 00001011	00101	101	0000 1011	MOV R5, #11
00011 101 00000110	00011	101	0000 0110	SUB R5, #6
00000 000 00000000	00000	000	0000 0000	HLT

```
MOV R3, #7
ADD R3, #15
MOD R3, #4
MOV R5, #11
SUB R5, #6
HLT
```

Hoofdstuk 5

5.1 (3p)

a. Een voorspelling -> Ik reflecteert er later op.

b. Reflectie

c. $0x294D = 0010\ 1001\ 0100\ 1101 = 00101\ 001\ 0100\ 1101 = \text{MOV } R1, \#7$

d. plaats het direct in het geheugen. **DAT** is de mnemonic om direct gegevens in het geheugen te zetten.

e. $4365 = 00010\ 001\ 0000\ 1101 = \text{ADD } R1, \#13$

f. **DAT 0** wordt $0000\ 0000\ 0000\ 0000$ en dat is de **HLT** instructie

g. $0x5104 = 0101\ 0001\ 0000\ 0100 = 01010\ 001\ 0000\ 0100 = \text{MUL } R1, \#4$ *Opmerking hier zit een mogelijke onduidelijkheid, want ik moest zelf flink zoeken om de MUL met immediate value te vinden.

Misschien nog toevoegen aan

h.

```
MOV R1, #7
ADD R1, #13
HLT
MUL R1, #4
```

5.2

a. **DAT 7** plaatst de waarde 7 aan het begin van het programma (positie 0)

LDR R1, 0 plaatst de waarde op geheugenplaats 0 in R1.

ADD R1, #23 tel er 23 bij op.

STR R1, 0 plaats de waarde van R1 weer op geheugenplaats 0

HLT stopt het programma

Na afloop van het draaien van dit programma staat de waarde 30 op geheugenplaats 0.

b. -

c. Bijvoorbeeld de onderstaande code. Het is belangrijk dat de directe adressering van de **LDR** en **STR** instructies op het juiste geheugenadres terecht komen.

```
LDR R1, 4
ADD R1, #23
STR R1, 4
HLT
DAT 7
```

5.3

```

LDR R1, 11 // laad de eerste operand (getal P) in R1    0
LDR R2, 12 // laad de tweede operand (getal Q) in R2    1
MUL R1, R2 // opdracht a                                2
STR R1, 15 // opslaan van antwoord A in het geheugen     3
LDR R2, 13 // laad de tweede operand (getal R) in R2    4
ADD R1, R2 // opdracht b                                5
STR R1, 16 // opslaan van antwoord B in het geheugen     6
LDR R2, 14 // laad de derde operand (getal S) in R2     7
SUB R2, R1 // opdracht c                                8
STR R2, 17 // opslaan van antwoord C in het geheugen     9
HLT // start van de datasectie                           10
DAT 7 // getal P (regel 8)                               11
DAT 2 // getal Q (regel 9)                               12
DAT 11 // getal R (regel 10)                             13
DAT 45 // getal S                                         14
DAT 0 // de plek voor antwoord A (regel 12)              15
DAT 0 // de plek voor antwoord B (regel 13)              16
DAT 0 // de plek voor antwoord C                         17

```

Het hernummeren van de directe referentie naar de data plekken is belangrijk!

5.4

Opletten:

- wordt de waarde goed berekend?
- wordt de waarde weer terug gezet op de juiste plek?

a.

```

// textsection
LDR R1, brightness // laad de brightness in R1
ADD R1, #10         // tel 10 op bij de brightness
STR R1, brightness // sla het resultaat weer op in het geheugen
HLT
// datasection
brightness: DAT 50

```

b.

```

// textsection
LDR R1, balance // laad de balance in R1
LDR R2, amount  // laad amount in R2
SUB R1, R2       // R1 = R1 - R2
STR R1, balance  // balance = balance - amount
HLT

```

```
// datasection
balance:   DAT 100
amount:    DAT 15
```

c.

```
// textsection
LDR R1, speed_y    // laad speed_y in R1
LDR R2, gravity     // laad gravity in R2
SUB R1, R2          // R1 = R1 - R2
STR R1, speed_y     // speed_y = speed_y - gravity
LDR R2, y           // laad y in R2
ADD R2, R1          // R2 = R2 + R1
STR R2, y           // y = y + speed_y
HLT

// datasection
y:      DAT 34
speed_y: DAT 0
gravity: DAT 2
```

5.5

a.

```
MOV R1, #0x43
OUT R1, 7
HLT
```

b.

```
MOV R1, #135
OUT R1, 4
HLT
```

c.

```
MOV R1, #16
OUT R1, 6
HLT
```

d.

```
MOV R1, #0x19
OUT R1, 4
HLT
```

e.

```
MOV R1, #0x65    // 'e' in R1
OUT R1, 7        // druk 'e' af
MOV R1, #0x6C    // 'l' in R1
OUT R1, 7        // druk 'l' af
MOV R1, #0x66    // 'f' in R1
OUT R1, 7        // druk 'f' af
HLT
```

f.

```
MOV R1, #0x34    // '4' in R1
OUT R1, 7        // druk '4' af
MOV R1, #0x2B    // '+' in R1
OUT R1, 7        // druk '+' af
MOV R1, #0x38    // '8' in R1
OUT R1, 7        // druk '8' af
MOV R1, #0x3D    // '=' in R1
OUT R1, 7        // druk '=' af
MOV R1, #4       // zet de waarde 4 in R1
ADD R1, #8       // tel er 8 bij op
OUT R1, 4        // druk de waarde decimaal af
```

5.6

```
INP R1, 2    // Haal invoer
OUT R1, 7    // druk af al ASCII-karakter
HLT
```

5.7

De eerste regel mag met **LDR** of met **INP**.

- Invoer 'veilig' stellen (1p)
- AND herhaaldelijk inzetten (1p)
- geen DIV bij laatste bit (1p)
- correcte uitvoer (1p)

```

    LDR R1, invoer    // haal invoer op
    MOV R4, R1        // sla de invoer op
    MOV R2, #8        //
    DIV R1, R2        // Deel invoer door 8
    AND R1, #1        // Zorg dat er zeker 0 of 1 staat
    OUT R1, 4         // print getal
    MOV R1, R4        // haal invoer opnieuw op
    MOV R2, #4        //
    DIV R1, R2        // Deel invoer door 4
    AND R1, #1        // Zorg dat er zeker 0 of 1 staat
    OUT R1, 4         // print getal
    MOV R1, R4        // haal invoer opnieuw op
    MOV R2, #2        //
    DIV R1, R2        // Deel invoer door 2
    AND R1, #1        // Zorg dat er zeker 0 of 1 staat
    OUT R1, 4         // print getal
    MOV R1, R4        // haal invoer opnieuw op
    AND R1, #1        // Zorg dat er zeker 0 of 1 staat
    OUT R1, 4         // print getal
    HLT
invoer: DAT 12

```

5.8

```

// textsection
    LDR R1, gemeten_snelheid
    LDR R2, meet_correctie
    SUB R1, R2                // gemeten_snelheid -= meet_correctie
    LDR R2, maximale_snelheid
    SUB R1, R2                // km_tehard = gemeten_snelheid -
maximale_snelheid
    MUL R1, #7                // km_tehard * 7
    ADD R1, #18               // km_tehard * 7 + 18 = boetebedrag
    LDR R2, admin_kosten
    ADD R1, R2                // totaal = boetebedrag + admin_kosten
    OUT R1, 4
    HLT
// datasection
gemeten_snelheid:  DAT 78
meet_correctie:    DAT 3
maximale_snelheid: DAT 70
admin_kosten:      DAT 9

```

De administratiekosten mogen gegokt worden (of die zijn juist die 18 euro). Administratiekosten zijn opgezocht op de site van CJIB.

5.9

Instructie	Formaat	Funcitie	Argumenten	Machinetaal
ADD R3, R5	B3, op = 0110	ADD, func = 000	Rd = 011, Rs = 011, Rb = 101	0110 00 011 011 101
MUL R7, R2	B10, op = 011101	MUL, func = 1111	Rsd = 111, Rb = 010	0111 0111 1111 1010
MOD R4, #5	A, op = 00	MOV, func = 001	Rsd = 100, #imm8 = 0000 0101	0000 1100 0000 0101
SUB R3, R2, R1	B3, op = 0110	SUB, func = 001	Rd = 011, Rs = 010, Rb = 001	0110 0010 1101 0001
MUL R7, #2	B1, op = 010	MUL, func = 10	Rsd = 111, #imm8 = 0000 0010	0101 0111 0000 0010
AND R3, R7	B3, op = 0110	AND, func = 010	Rd = 011, Rs = 011, Rb = 111	0110 0100 1101 1111

Gecontroleerd met de RISC simulator

Hoofdstuk 6

6.1

- a. De processor springt terug naar de MUL instructie. Inzicht: begin met 0 te tellen
- b. Na 10 herhalingen: $2^{10} = 1024$
- c. Na 15 herhalingen: $2^{15} = 32.768$
- d. Na 20 herhalingen: $2^{20} = 1.048.576$
- e. Bij 16384 springt R1 over naar -32768. Vanwege twee complements notatie. Daarna naar 0. En blijft op 0.
- f. 0x8000

6.2

```
begin:  INP R1, 2      // lees de input
        CMP R1, #30   // is de input gelijk aan 30?
        BNE begin     // niet gelijk -> begin
einde:  MOV R1, #0     // print het getal 0
        OUT R1, 4
        HLT
```

6.3

```
// text
                INP R1, 2          // batterij-niveau opvragen via input
                CMP R1, #10
                BLT if_batlow      // if bat<10
                BRA skip_batlow    // if not skip_batlow
if_batlow:      LDR R2, warning    // uitroepteken inladen
                OUT R2, 7          // uitroepteken weergeven
skip_batlow:    HLT
// data
warning: DAT 0x21
```

6.4

```
// text
begin:          INP R1, 2          // lees het te raden getal
                MOV R3, #0         // aantal pogingen = 0
nog_een_keer:   INP R2, 2          // lees de gok
                ADD R3, #1         // poging +1
                CMP R2, R1         // vergelijk gok en getal
                BLT kleiner        // getal is kleiner
                BGT groter         // getal is groter
                BEQ gelijk         // getal is gelijk
kleiner:        MOV R4, #0x3C     // < teken inladen
                OUT R4, 7          // < teken weergeven
                BRA nog_een_keer   // nog een raadpoging
groter:         MOV R4, #0x3E     // > teken inladen
                OUT R4, 7          // > teken weergeven
                BRA nog_een_keer   // nog een raadpoging
gelijk:         OUT R3, 4          // druk het aantal pogingen af
                HLT
```

6.5

De sturings logica is niet ingewikkeld aan deze opdracht. Wat de opdracht ingewikkeld maakt is het afdrukken van de tekst. Interessant is om te zien of leerlingen een CRLF aan het einde van een tekst bedenken 😊

```
// text
vraag:          INP R1, 2          // lees de ingevoerde pincode
                LDR R2, pincode    // laad de juiste pincode
                CMP R1, R2         // vergelijk de codes
                BEQ pincode_juist  // de pincode is juist
                BNE pincode_onjuist // de pincode is onjuist
pincode_juist:  MOV R1, #0x4F      // laad '0'
                OUT R1, 7          // '0' weergeven
                MOV R1, #0x4B      // laad 'K'
                OUT R1, 7          // 'K' weergeven
```



```

                BRA vraag                // ga weer terug
pincode_onjuist:MOV R1, #0x57            // laad 'W'
                OUT R1, 7                // 'W' weergeven
                MOV R1, #0x52            // laad 'R'
                OUT R1, 7                // 'R' weergeven
                MOV R1, #0x4F            // laad 'O'
                OUT R1, 7                // 'O' weergeven
                MOV R1, #0x4E            // laad 'N'
                OUT R1, 7                // 'N' weergeven
                MOV R1, #0x47            // laad 'G'
                OUT R1, 7                // 'G' weergeven
                MOV R1, #0x20            // laad ' '
                OUT R1, 7                // ' ' weergeven
                MOV R1, #0x50            // laad 'P'
                OUT R1, 7                // 'P' weergeven
                MOV R1, #0x49            // laad 'I'
                OUT R1, 7                // 'I' weergeven
                MOV R1, #0x4E            // laad 'N'
                OUT R1, 7                // 'N' weergeven
                BRA vraag                // ga weer terug

// data
pincode: DAT 4367

```

6.6

```

// text
inp_max:      INP R1, 2                // max.score
              STR R1, max_score        // sla de max_score op
inp_score:    INP R1, 2                // de score in R1
              MUL R1, #10              // bereken het cijfer
              LDR R2, max_score        // laad de max_score
              DIV R1, R2               // het cijfer staat in R1
              CMP R1, #100             // is het een maximaal cijfer?
              BEQ druk_10              // druk een 10,0 af
              MOV R3, R1               // zet het cijfer in R3
              MOV R4, #10              // 
              DIV R3, R4               // deel door 10
              OUT R3, 4                // druk het getal af
              MOV R3, #0x2C            // laad ',' teken
              OUT R3, 7                // druk ',' teken af
              MOV R3, R1               // zet het cijfer in R3
              MOD R3, #10              // pik het cijfer achter de komma op
              OUT R3, 7                // druk dit cijfer af
              BRA einde
druk_10:      MOV R3, #10              // zet 10 in R3
              OUT R3, 4                // druk 10 af
              MOV R3, #0x2C            // laad ',' teken
              OUT R3, 7                // druk ',' teken af
              MOV R3, #0x30            // laad '0' teken
              OUT R3, 7                // druk '0' teken af
einde:      HLT

```

```
// data  
max_score: DAT 0
```